

树莓派与机械臂

不能把 .xml 文件内容直接 `serial.write()` 给机械臂。XML 是原厂上位机保存的动作组文本，需要先在树莓派解析为若干“舵机位置 + 时间”的动作帧，再通过 USB 串口发送控制帧。

当前的 LeArm 链路应是：

```
XML 动作组 -> 树莓派解析 -> 0x03 多舵机运动帧 -> USB 串口 9600 8N1 -> LeArm
STM32 -> 6 路总线舵机
```

官方协议的多舵机运动命令是：

```
55 55 Length 03 count time_lo time_hi id pos_lo pos_hi ...
```

动作组 0x06 只能运行已经下载到控制板 Flash 的动作组。官方串口协议

([https://docs.hiwonder.com/projects/Bus-Servo-Controller/en/latest/docs/2.Bus Servo Controller Secondary Development.html](https://docs.hiwonder.com/projects/Bus-Servo-Controller/en/latest/docs/2.Bus%20Servo%20Controller%20Secondary%20Development.html))

你们的 XML，例如 PICK_APPROACH.xml，已是可直接转换的格式：

```
<Move>#1 P226 #2 P500 #3 P177 #4 P129 #5 P408 #6 P500</Move>
<Time>T1500</Time>
```

这表示发送一次六舵机同步运动：ID 1..6 分别到对应位置，运动时长 1500 ms。

需要的 Python 库/函数：

- `pyserial`: `serial.Serial(...)`、`ser.write(frame)`、`ser.read(...)`
- 标准库 `xml.etree.ElementTree`: `ET.parse()`、`root.findall()`
- 标准库 `re`：解析 `<Move>` 中的 #ID P位置

示例核心逻辑如下：

```
import re
import time
import xml.etree.ElementTree as ET
import serial

from robogame2026_auto.arm.arm_protocol import build_servo_move

def load_xml_action(xml_path: str):
    root = ET.parse(xml_path).getroot()
    frames = []

    for table in root.findall("./Table"):
        move_text = None
```

```

        for child in table:
            if child.tag == "Move":
                move_text = child.text or ""
            elif child.tag == "Time" and move_text is not None:
                duration_ms = int(re.search(r"T(\d+)", child.text or
""))
                positions = {
                    int(servo_id): int(position)
                    for servo_id, position in re.findall(r"#
(\d+)\s+P(\d+)",
                    move_text)
                }
                frames.append((positions, duration_ms))
                move_text = None

        return frames

port = "/dev/serial/by-id/usb-实际设备名"
frames = load_xml_action("arm_action_file/PICK_APPROACH.xml")

with serial.Serial(port, baudrate=9600, timeout=0.2, write_timeout=0.2) as
ser:
    for positions, duration_ms in frames:
        ser.write(build_servo_move(positions, duration_ms))
        ser.flush()
        time.sleep(duration_ms / 1000.0)

```